CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[Section]
System Integration and Architecture

System Design Document (SDD)

Project Title: Brainbox:

Prepared By: Gako, Joana Carla D.

Version: 3.0

Date: February 27, 2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	February 20, 2026	Gako, Joana Carla D.	Created initial outline.	Draft
0.2	March 7, 2026	Gako, Joana Carla D.	Added feature design.	Review
0.3	March 22, 2026	Gako, Joana Carla D.	Updated API and database notes.	Review
0.4	April 8, 2026	Gako, Joana Carla D.	Added system architecture section.	Revised
0.5	April 28, 2026	Gako, Joana Carla D.	Added UI/UX and plan sections.	Revised
1.0	May 1, 2026	Gako, Joana Carla D.	Finalized SDD content.	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY

1.1 Project Overview

BrainBox is a multi-platform study application that helps users create notebooks, organize study materials, recover notebook content through version history, generate quizzes and flashcards, maintain playlists and playback queues, and use AI-assisted notebook features. The system includes a Spring Boot backend API, a React/Vite web application, and an Android/Kotlin mobile application integrated to provide a consistent study experience across platforms.

1.2 Objectives

1. Provide secure account access for BrainBox users.
2. Allow users to create, organize, edit, review, export, version, and restore notebooks.
3. Support quiz and flashcard study workflows generated from user learning materials.
4. Provide playlist, queue, and playback features for audio-based study.
5. Support mobile study screens and playback controls.
6. Maintain consistent web and mobile user experiences.

1.3 Scope

Included Features:

- User registration, login, logout, token refresh, email verification, password reset, and Google sign-in.
- User profile retrieval, profile update, password change, settings modal, and AI provider settings access.
- Notebook creation, listing, content editing, review tracking, deletion, version history, version preview, manual snapshots, stale-version conflict handling, and version restore.
- Category creation, listing, lookup, and deletion.
- AI configuration, selected AI provider management, AI query, and AI conversations.
- Quiz creation, editing, listing, studying, deletion, and attempt recording.

- Flashcard deck creation, editing, listing, studying, deletion, and attempt recording.
- Playlist creation, notebook queue management, reorder, deletion, and current index updates.
- Playback queue management and web/mobile playback controls.
- Web notebook editor formatting, outline navigation, review mode, and PDF/Word/text export.

Excluded Features:

- Real-time collaborative editing.
- Payment or subscription processing.
- Public content sharing platform.
- Production analytics dashboard.
- Admin management screens.
- Offline mobile study mode.

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the BrainBox system. It provides detailed requirements, architectural decisions, API contracts, database design, UI/UX design, and implementation plan to guide development and ensure all components integrate properly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: BrainBox
Domain: EdTech / Personal Knowledge Management
Primary Users:

1. Students, self-directed learners.
2. Professionals studying reference material.

Problem Statement: Students need a focused way to create notes, organize learning materials, generate study activities, review content through quizzes and flashcards, and continue studying across web and mobile devices.
Solution: BrainBox provides a notebook-centered study platform with authentication, profile/settings management, notebook management, version history, AI-assisted study support, quizzes, flashcards, playlists, playback queue management, and mobile study/playback support.

2.2 Core User Journeys

Journey 1: Account Access and Session Recovery

1. User opens the web or mobile application.
2. User registers with username, email, and password, or signs in with existing credentials.
3. System sends or validates verification/reset information where required.
4. System issues access and refresh tokens after successful login.
5. User reaches the dashboard/home shell.
6. If the access token expires, the client uses the refresh-token endpoint before asking the user to sign in again.

Journey 2: Profile and Settings Management

1. User opens the Profile page from the authenticated navigation.
2. System displays account identity details such as username, email, and joined date where available.
3. User opens Settings from the profile actions or profile dropdown.
4. User edits profile information in the Profile tab and saves changes.
5. User changes the account password in the Password tab.
6. User configures AI provider information in the AI Provider tab.
7. User may log out after confirming the logout action.

Journey 3: Notebook Authoring, Version History, and Restore

1. User opens the dashboard or library.
2. User creates a notebook and optionally assigns it to a category.
3. User edits content in the TipTap editor with formatting, table, math, outline, and page-layout controls.
4. User saves content; the backend persists the notebook and records a version snapshot.
5. User opens Version History, filters versions by date, previews an older version, and compares it in the preview overlay.
6. User restores a selected version when needed; the backend updates the notebook and returns the restored content.
7. If a stale base version is submitted, the backend returns a conflict response with the latest notebook data.

Journey 4: AI-Assisted Notebook and Study Material Creation

1. User opens a notebook in edit or review mode.
2. User opens AI settings and selects or saves an AI configuration.
3. User sends a prompt, selected text, or multiple AI highlight targets to the AI endpoint.
4. System stores or updates the AI conversation for the notebook.
5. User reviews the AI proposal, accepts useful edits, and saves the result.
6. User can create or refine quizzes and flashcards from notebook content.

Journey 5: Quiz and Flashcard Study

1. User opens the quizzes or flashcards section on web or mobile.
2. User creates, edits, searches, or opens existing study material tied to a notebook.
3. User answers quiz questions or studies flashcard prompts.
4. User submits the attempt.
5. Backend records the score or mastery result and returns updated study data.

Journey 6: Playlist and Audio Playback

1. User creates or opens a playlist.
2. User adds notebooks, removes notebooks, reorders items, and updates the current index.
3. User selects the playlist as the current playback queue or adds notebooks directly to the queue.
4. User starts playback from the web player bar or Android global playbar.
5. User controls play, pause, skip, loop, shuffle, speech rate, queue overlay, and resume position.
6. Playback marks notebooks as reviewed where supported by the client flow.

2.3 Feature List (MoSCoW)

MUST HAVE

1. Authentication and protected API access.
2. Profile and settings management.
3. Notebook CRUD, content save, review state, editor formatting, version history, version preview, and version restore.
4. Category management.
5. Quiz and flashcard management with attempt recording.
6. Playlist and playback queue management.
7. Web and mobile access to major study workflows.
8. Mobile playback controls and background playback service.

SHOULD HAVE

1. AI configuration and notebook AI assistance.
2. Editor export to PDF/print, Word .docx, and text .txt.

COULD HAVE

1. Expanded study analytics.
2. Additional AI prompt modes.
3. Additional export formats.

WON'T HAVE

1. Payment processing.
2. Real-time collaborative editing.
3. Public content sharing platform.
4. Offline mobile study mode.
5. Admin management workflows.

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Username, Email, Password, Confirm Password, Verification Code
- Validation: Email format, required fields, password confirmation, valid reset code, unique username/email
- **API Endpoints:** POST /api/auth/register, GET /api/auth/verify-email, POST /api/auth/login, POST /api/auth/logout, POST /api/auth/refresh-token, POST /api/auth/tokens/refresh, POST /api/auth/forgot-password, POST /api/auth/verify-code, POST /api/auth/reset-password, POST /api/auth/google
- **Security:** JWT tokens, refresh tokens, password hashing, protected routes

Feature: Profile and Settings Management

- **Screens:** Profile, Settings Modal
- **Fields:** Username, Email, Current Password, New Password, Confirm Password, AI Config Name, AI Model, Proxy URL, API Key
- **Validation:** Authenticated user required, valid profile values, valid current password for password changes, matching new password confirmation, valid AI provider values when configuring AI
- **API Endpoints:** GET /api/user/me, GET /api/users/me, PUT /api/user/me, PUT /api/users/me, POST /api/user/me/change-password, POST /api/users/me/password
- **Settings Functions:** View account summary, edit profile, change password, open AI Provider settings, save/select/delete AI provider configuration, and log out after confirmation
- **Security:** User can only access and update their own profile and settings

Feature: Notebook Management

- **Screens:** Dashboard, Library, Notebook Editor, Version History
- **Fields:** Notebook Title, Content, Category, Review Status, Version, Base Version, Client Mutation ID
- **Validation:** Authenticated user required, notebook ownership, valid notebook UUID, category ownership, stale version conflict handling
- **API Endpoints:** POST /api/notebooks, GET /api/notebooks, GET /api/notebooks/recently-edited, GET /api/notebooks/recently-reviewed, GET /api/notebooks/{uuid}, PUT /api/notebooks/{uuid}, PUT /api/notebooks/{uuid}/content, PATCH /api/notebooks/{uuid}/review, PATCH /api/notebooks/update-review/{uuid}, DELETE /api/notebooks/{uuid}
- **Version Endpoints:** GET /api/notebooks/{notebookUuid}/versions, GET /api/notebooks/{notebookUuid}/versions/{versionId}, POST /api/notebooks/{notebookUuid}/versions, POST /api/notebooks/{notebookUuid}/versions/{versionId}/restore
- **Editor Functions:** Format rich text, create tables and math content, use outline navigation, enter review mode, export PDF/print, export Word .docx, export text .txt, preview versions, and restore versions
- **Persistence:** Database storage per user with notebook version snapshots and notebook mutation records for idempotent write handling

Feature: Category Management

- **Screens:** Library, Category Filter, Category Actions
- **Fields:** Category Name
- **Validation:** Authenticated user required, category ownership, non-empty category name
- **API Endpoints**: GET /api/categories, POST /api/categories, GET /api/categories/{id}, DELETE /api/categories/{id}
- **Process:** Organize notebooks by category and update affected notebooks when a category is deleted

Feature: AI Assistance

- **Screens:** AI Config Panel, Notebook Editor AI Sidebar, AI Conversation Panel
- **Fields:** Config Name, Model, Proxy URL, API Key, Prompt, Conversation Title, Notebook UUID, Selected Text, AI Selection Targets, Mode
- **Validation:** Authenticated user required, selected AI configuration, valid config values
- **API Endpoints:** GET /api/ai/config, GET /api/ai/configs/selected, GET /api/ai/config/list, GET /api/ai/configs, PUT /api/ai/config, PUT /api/ai/configs, PUT /api/ai/config/{id}/select, PUT /api/ai/configs/{id}/selected, DELETE /api/ai/config/{id}, DELETE /api/ai/configs/{id}, POST /api/ai/query, GET/POST/PUT/DELETE /api/ai/conversations

- **Functions**: AI-assisted notebook editing, review-mode assistance, selected-text prompts, multi-highlight selection prompts, quiz generation, flashcard generation, and per-notebook conversation persistence

Feature: Quiz Management

- **Screens:** Quizzes, Create Quiz, Edit Quiz, Quiz Study, Quiz Results
- **Fields:** Quiz Title, Description, Difficulty, Notebook UUID, Question Type, Question Text, Options, Correct Answer, Client Mutation ID
- **Validation:** Authenticated user required, quiz ownership, valid questions and answers
- **API Endpoints:** POST /api/quizzes, GET /api/quizzes, GET /api/quizzes/{uuid}, PUT /api/quizzes/{uuid}, DELETE /api/quizzes/{uuid}, POST /api/quizzes/{uuid}/attempts
- **Functions:** Create quizzes from notebooks, study questions on web or mobile, record scores, and show attempt/best-score information

Feature: Flashcard Management

- **Screens:** Flashcards, Create Deck, Edit Deck, Flashcard Study, Flashcard Results
- **Fields:** Deck Title, Description, Notebook UUID, Card Front, Card Back, Mastery, Client Mutation ID
- **Validation:** Authenticated user required, deck ownership, valid card front/back values
- **API Endpoints:** POST /api/flashcards, GET /api/flashcards, GET /api/flashcards/{uuid}, PUT /api/flashcards/{uuid}, DELETE /api/flashcards/{uuid}, POST /api/flashcards/{uuid}/attempts
- **Functions:** Create flashcard decks from notebooks, study cards on web or mobile, record mastery, and show attempt/best-mastery information

Feature: Playlist and Playback Queue

- **Screens:** Playlists, Playlist Detail, Playback Queue, Mobile Playbar
- **Fields:** Playlist Title, Notebook Order, Current Index, Queue Items
- **Validation:** Authenticated user required, playlist ownership, notebook ownership, valid queue order
- **API Endpoints:** POST /api/playlists, GET /api/playlists, GET /api/playlists/{uuid}, PUT /api/playlists/{uuid}, DELETE /api/playlists/{uuid}, POST /api/playlists/{uuid}/notebooks, DELETE /api/playlists/{uuid}/notebooks/{notebookUuid}, PUT /api/playlists/{uuid}/reorder, PATCH /api/playlists/{uuid}/current-index, GET /api/queue, GET /api/playback-queues/current, POST /api/queue/notebooks, POST /api/playback-queues/current/notebooks, PUT /api/queue/playlist/{playlistUuid}, PUT /api/playback-queues/current/playlist/{playlistUuid}, DELETE /api/queue,

DELETE /api/playback-queues/current, PUT /api/queue/reorder, PUT /api/playback-queues/current/reorder

- **Functions:** Add notebooks, remove notebooks, reorder playlists, persist current index, start queue playback, skip items, control queue from the web player bar and Android global playbar

2.5 Acceptance Criteria

AC-1: Successful User Registration

- ➢ Given I am a new user
- ➢ When I enter a unique username, valid email, and valid password
- ➢ And confirm password matches
- ➢ And click "Create Account"
- ➢ Then my account should be created
- ➢ And I should be able to log in using the new account

AC-2: Successful User Login

- ➢ Given I am a registered user
- ➢ When I enter valid login credentials
- ➢ And click "Sign In"
- ➢ Then I should receive an authenticated session
- ➢ And I should be redirected to the dashboard or mobile home shell

AC-3: Profile and Settings Management

- ➢ Given I am logged in as an authenticated user
- ➢ When I open the Profile page
- ➢ Then I should see my account information
- ➢ When I open Settings
- ➢ Then I should be able to update profile details, change password, and configure the AI provider from the available settings tabs

AC-4: Notebook Creation and Editing

- ➢ Given I am logged in as an authenticated user
- ➢ When I create a notebook with valid details
- ➢ And edit and save notebook content
- ➢ Then the notebook should appear in my library
- ➢ And the saved content should be available when I reopen the notebook
- ➢ And a notebook version should be recorded

AC-5: Notebook Version Preview and Restore

- Given I am logged in as an authenticated user
- And my notebook has saved versions
- When I open Version History
- And select a previous version
- Then I should see a preview of that version
- When I restore the selected version
- Then the notebook should return the restored content
- And a later save should continue the notebook version sequence

AC-6: Notebook Version Conflict

- Given I am editing an outdated notebook version
- When I submit a save with a stale base version
- Then the backend should reject the save with a conflict response
- And the response should include the latest notebook data.

AC-7: AI-Assisted Editing

- Given I am logged in and have a selected AI configuration
- When I select text or AI highlight targets in a notebook
- And submit an AI query
- Then the system should return an AI response
- And the conversation should be available for the notebook

AC-8: Quiz Study Flow

- Given I am logged in as an authenticated user
- And I have an existing quiz
- When I answer the quiz questions
- And submit the quiz attempt
- Then my score should be recorded
- And the quiz result should be shown

AC-9: Flashcard Study Flow

- Given I am logged in as an authenticated user
- And I have an existing flashcard deck
- When I study the cards
- And record my mastery result
- Then the flashcard attempt should be saved
- And the updated mastery result should be shown

AC-10: Playlist Playback Flow

- Given I am logged in as an authenticated user

- ➢ When I create a playlist
- ➢ And add notebooks to the playlist
- ➢ And start the playback queue
- ➢ Then the selected notebooks should appear in playback order
- ➢ And I should be able to skip between queue items

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- Web routes should support lazy-loaded pages where appropriate and provide a route error fallback for load failures.
- Study lists should use pagination where implemented.
- Mobile playback should split long notebook text into manageable speech chunks.
- Mobile playback should retain the current audio position for resume continuity.
- Notebook content saves should use version checks to avoid overwriting newer data with stale client data.
- Repeated notebook mutations and study attempts should be safe where client mutation IDs are supported.
- Backend CRUD and study operations should complete within acceptable demo time under normal development conditions.

3.2 Security Requirements

- Protected endpoints require authenticated access.
- User-owned records must be scoped to the authenticated owner.
- Passwords must be stored using one-way encryption or hashing.
- Refresh tokens must include expiry information.
- AI keys and email credentials must not be exposed through client source code.

3.3 Compatibility Requirements

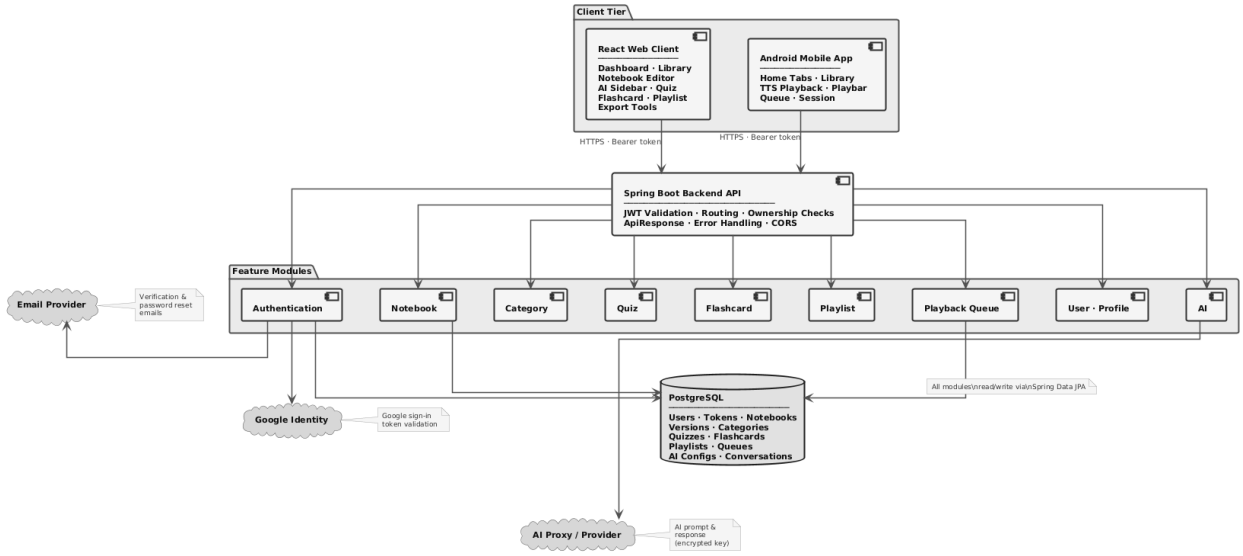
- The web application must run in modern browsers supported by React and Vite.
- The Android application must support Android API level 24 and higher.

3.4 Usability Requirements

- The web application must provide clear navigation for dashboard, library, quizzes, flashcards, playlists, and profile.
- The notebook editor must provide visible controls for editing, AI assistance, export, outline navigation, review mode, and version history.
- The mobile application must provide touch-friendly navigation, study screens, and playback controls.
- The system must clearly communicate version conflict, missing record, unauthorized, and external service failure states.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 21, Spring Boot, Spring Security, Spring Data JPA, Maven
- **Database:** PostgreSQL (Supabase)
- **Web Frontend:** React, Vite, React Router, TanStack Query, Tiptap, npm
- **Mobile:** Kotlin, Android Jetpack Compose, Retrofit, Media3, Gradle
- **External Services:** Email service, Google sign-in, AI proxy provider
- **Deployment:** ackend API server, web build hosting, Android APK

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api
Format	JSON for all requests and responses.
Authentication	Bearer token in the Authorization header for protected endpoints.
Response Structure	{ "success": true, "data": {},

	"error": null, "timestamp": "2026-05-01T00:00:00Z" }

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/api/auth/register
HTTP Request Method	POST
Format	JSON for all requests and responses
Authentication	None
Request Payload	{ "username": "<username>", "email": "<email>", "password": "<password>" }
Response Structure	{ "success": true, "data": null, "error": null, "timestamp": "2026-05-01T00:00:00Z" }

User Login

Description	User Login
API URL	/api/auth/login
HTTP Request Method	POST
Format	JSON for all requests and responses
Authentication	None
Request Payload	{ "username": "<username_or_email>", "password": "<password>" }
Response Structure	{ "success": true, "data": { "user": { "username": "<username>", "email": "<email>", "role": "<role>" }, "accessToken": "<token>",

	"refreshToken": "<token>" }, "error": null, "timestamp": "2026-05-01T00:00:00Z" }

Additional Endpoint Groups

Authentication Endpoint Table

Method	Endpoint	Authentication	Request / Parameters	Response / Result
POST	/api/auth/register	None	username, email, password	Creates account; returns success with data: null.
GET	/api/auth/verify-email	None	Query parameter: token	Verifies email token and returns redirect/void response.
POST	/api/auth/forgot-password	None	email	Sends password reset code; returns success with data: null.
POST	/api/auth/verify-code	None	email, code	Validates reset code; returns verification result.
POST	/api/auth/reset-password	None	token, newPassword	Updates account password; returns success with data: null.
POST	/api/auth/login	None	username, password	Returns accessToken

				and refreshToken.
POST	/api/auth/logout	None	refreshToken	Invalidates refresh token; returns success with data: null.
POST	/api/auth/refresh-token; alias /api/auth/tokens/refresh	None	refreshToken	Returns refreshed login token data.
POST	/api/auth/google	None	idToken, accessToken	Authenticates with Google and returns login token data.

Profile Endpoint Table

Method	**Endpoint**	**Authentication**	**Request / Parameters**	**Response / Result**
GET	/api/user/me; alias /api/users/me	User token	Authenticated userId request attribute	Returns authenticated user profile.
PUT	/api/user/me; alias /api/users/me	User token	username, email	Updates profile and returns updated profile data.
POST	/api/user/me/change-password; alias /api/users/me/password	User token	currentPassword, newPassword	Changes password and returns success with data: null.

Notebook Endpoint Table

Method	Endpoint	Authentication	Request / Parameters	Response / Result
POST	/api/notebooks	User token	title, categoryId, content, baseVersion, clientMutationId	Creates notebook and returns full notebook data.
GET	/api/notebooks	User token	Authenticated userId	Returns notebook overview list.
GET	/api/notebooks/recently-edited	User token	Authenticated userId	Returns recently edited notebook overview list.
GET	/api/notebooks/recently-reviewed	User token	Authenticated userId	Returns recently reviewed notebook overview list.
GET	/api/notebooks/{uuid}	User token	Notebook UUID	Returns full notebook data.
PUT	/api/notebooks/{uuid}	User token	Notebook UUID plus notebook request fields	Updates notebook metadata/content and returns full notebook data.
PATCH	/api/notebooks/update-review/{uuid}; alias /api/noteboo	User token	Notebook UUID plus review mutation data	Updates review state and returns success with data: null.

	ks/{uuid}/review			
PUT	/api/notebooks/{uuid}/content	User token	content, baseVersion, clientMutationId	Saves notebook content and returns full notebook data.
DELETE	/api/notebooks/{uuid}	User token	Notebook UUID plus optional mutation data	Deletes notebook and returns success with data: null.
GET	/api/notebooks/{notebookUuid}/versions	User token	Notebook UUID	Returns notebook version list.
GET	/api/notebooks/{notebookUuid}/versions/{versionId}	User token	Notebook UUID, version ID	Returns selected notebook version.
POST	/api/notebooks/{notebookUuid}/versions	User token	content	Creates notebook version snapshot and returns version data.
POST	/api/notebooks/{notebookUuid}/versions/{versionId}/restore	User token	Notebook UUID, version ID	Restores version and returns full notebook data.

Category Endpoint Table

Method	**Endpoint**	**Authentication**	**Request / Parameters**	**Response / Result**
GET	/api/categories	User token	Authenticated userId	Returns user category list.

POST	/api/categories	User token	name	Creates category and returns category data.
GET	/api/categories/{id}	User token	Category ID	Returns category data.
DELETE	/api/categories/{id}	User token	Category ID, optional deleteNotebooks	Deletes category and returns success with data: null.

AI Endpoint Table

Method	**Endpoint**	**Authentication**	**Request / Parameters**	**Response / Result**
POST	/api/ai/query	User token	query, notebookUuid, conversationHistory, selectedText, aiSelections, selectionMode, mode	Returns AI-generated response.
GET	/api/ai/config; alias /api/ai/configs/selected	User token	Authenticated userId	Returns selected AI configuration.
GET	/api/ai/config/list; alias /api/ai/configs	User token	Authenticated userId	Returns AI configuration list.
PUT	/api/ai/config; alias /api/ai/configs	User token	id, name, model, proxyUrl, apiKey	Creates or updates AI configuration.

PUT	/api/ai/config/{id}/select; alias /api/ai/configs/{id}/selected	User token	AI config ID	Selects AI configuration and returns config data.
DELETE	/api/ai/config/{id}; alias /api/ai/configs/{id}	User token	AI config ID	Deletes AI configuration and returns success with data: null.
GET	/api/ai/conversations	User token	Optional filters handled by controller	Returns AI conversation list.
POST	/api/ai/conversations	User token	notebookUuid, mode, title, messages	Saves conversation and returns conversation data.
PUT	/api/ai/conversations/{uuid}	User token	Conversation UUID plus conversation request fields	Updates conversation and returns conversation data.
DELETE	/api/ai/conversations/{uuid}	User token	Conversation UUID	Deletes conversation and returns success with data: null.

Quiz Endpoint Table

Method	**Endpoint**	**Authentication**	**Request / Parameters**	**Response / Result**
POST	/api/quizzes	User token	title, description, difficulty,	Creates quiz and returns quiz data.

			notebookUuid, questions	
GET	/api/quizzes	User token	Authenticated userId	Returns quiz list.
GET	/api/quizzes/{uuid}	User token	Quiz UUID	Returns quiz data.
PUT	/api/quizzes/{uuid}	User token	Quiz UUID plus quiz request fields	Updates quiz and returns quiz data.
DELETE	/api/quizzes/{uuid}	User token	Quiz UUID	Deletes quiz and returns success with data: null.
POST	/api/quizzes/{uuid}/attempts	User token	score, clientMutationId	Records quiz attempt and returns updated quiz data.

Flashcard Endpoint Table

Method	**Endpoint**	**Authentication**	**Request / Parameters**	**Response / Result**
POST	/api/flashcards	User token	title, description, notebookUuid, cards	Creates flashcard deck and returns deck data.
GET	/api/flashcards	User token	Authenticated userId	Returns flashcard deck list.
GET	/api/flashcards/{uuid}	User token	Deck UUID	Returns flashcard deck data.
PUT	/api/flashcards/{uuid}	User token	Deck UUID plus flashcard request fields	Updates deck and returns deck data.

DELETE	/api/flashcards/{uuid}	User token	Deck UUID	Deletes deck and returns success with data: null.
POST	/api/flashcards/{uuid}/attempts	User token	mastery, clientMutationId	Records flashcard attempt and returns updated deck data.

Playlist Endpoint Table

Method	Endpoint	Authentication	Request / Parameters	Response / Result
POST	/api/playlists	User token	title	Creates playlist and returns playlist data.
GET	/api/playlists	User token	Authenticated userId	Returns playlist list.
GET	/api/playlists/{uuid}	User token	Playlist UUID	Returns playlist data.
PUT	/api/playlists/{uuid}	User token	Playlist UUID, title	Updates playlist and returns playlist data.
DELETE	/api/playlists/{uuid}	User token	Playlist UUID	Deletes playlist and returns success with data: null.
POST	/api/playlists/{uuid}/notebooks	User token	Playlist UUID, notebookUuid	Adds notebook and returns playlist data.
DELETE	/api/playlists/{uuid}/notebooks/{notebookUuid}	User token	Playlist UUID, notebook UUID	Removes notebook and returns playlist data.

PUT	/api/playlists/{uuid}/reorder	User token	Playlist UUID, notebookUuids	Reorders playlist and returns playlist data.
PATCH	/api/playlists/{uuid}/current-index	User token	Playlist UUID, current index value	Updates playlist current index and returns playlist data.

Playback Queue Endpoint Table

Method	Endpoint	Authentication	Request / Parameters	Response / Result
GET	/api/queue; alias /api/playback-queues/current	User token	Authenticated userId	Returns current playback queue.
POST	/api/queue/notebooks; alias /api/playback-queues/current/notebooks	User token	notebookUuid	Adds notebook to queue and returns queue data.
PUT	/api/queue/playlist/{playlistUuid}; alias /api/playback-queues/current/playlist/{playlistUuid}	User token	Playlist UUID	Selects playlist and returns queue data.
DELETE	/api/queue/notebooks/{notebookUuid}; alias /api/playback-queues/current/notebook	User token	Notebook UUID	Removes notebook and returns queue data.

	s/{notebookUuid}			
DELETE	/api/queue; alias /api/playback-queues/current	User token	Authenticated userId	Clears queue and returns success with data: null.
PATCH	/api/queue/current-index; alias /api/playback-queues/current/index	User token	Current index value	Updates queue current index and returns queue data.
PUT	/api/queue/reorder; alias /api/playback-queues/current/reorder	User token	notebookUuids	Reorders queue and returns queue

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	{
	"success": false,
	"data": null,
	"error": {
	"code": "BAD_REQUEST",
	"message": "Invalid request data",

	"details": null }, "timestamp": "2026-05-01T00:00:00Z" }

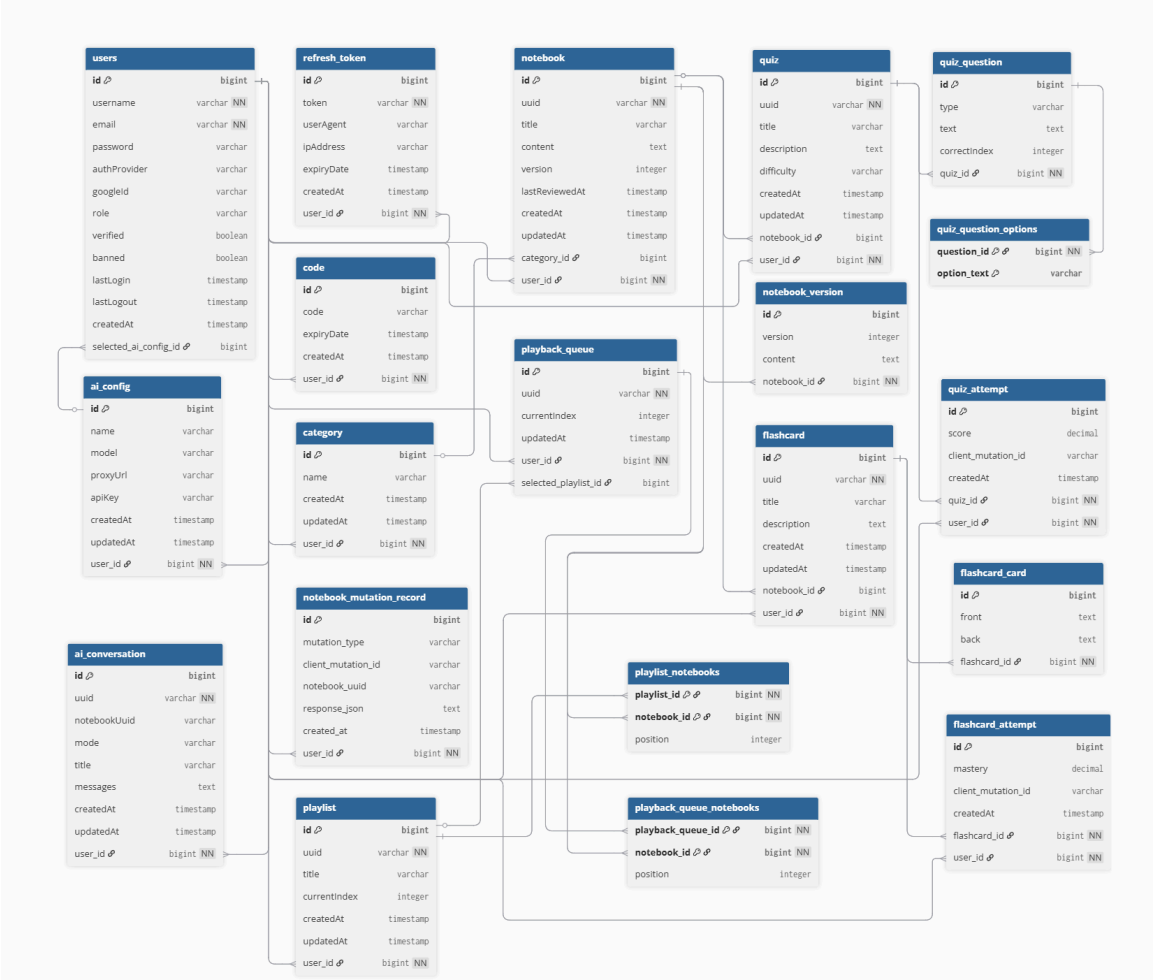
Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VERSION_CONFLICT",
	"message": "Notebook has been updated by another request",
	"details": {
	"latestNotebook": {
	"id": 12,
	"title": "Biology Review Notes",
	"version": 4
	}
	}
	},
	"timestamp": "2026-05-01T00:00:00Z"
	}

Common Error Codes

- BAD_REQUEST: Invalid input, invalid operation, or invalid request state.
- NOT_FOUND: Requested record does not exist.
- FORBIDDEN: Authenticated user is not allowed to access the requested resource.
- VERSION_CONFLICT: Notebook update uses an outdated version and must be retried with the latest data.
- INTERNAL_SERVER_ERROR: Unexpected server-side error.
- Unauthorized: Current authentication interceptor response for missing, invalid, or expired bearer tokens.

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

One-to-Many:

- User -> Notebooks. A user can own multiple notebooks.
- User -> Categories. A user can create multiple categories.
- User -> RefreshTokens. A user can have multiple refresh token sessions.
- User -> AiConfigs. A user can create multiple AI provider configurations.
- User -> AiConversations. A user can save multiple AI conversations by notebook UUID.
- Notebook -> NotebookVersions. A notebook can have multiple saved content snapshots.
- Quiz -> QuizQuestions. A quiz can contain multiple questions.
- Quiz -> QuizAttempts. A quiz can have multiple recorded attempts.

- Flashcard -> FlashcardCards. A flashcard deck can contain multiple cards.
- Flashcard -> FlashcardAttempts. A flashcard deck can have multiple recorded attempts.

Many-to-One:

- Notebook -> User. Each notebook belongs to one user.
- Notebook -> Category. A notebook may belong to one category.
- Quiz -> User. Each quiz belongs to one user.
- Quiz -> Notebook. A quiz may be associated with one notebook.
- Flashcard -> User. Each flashcard deck belongs to one user.
- Flashcard -> Notebook. A flashcard deck may be associated with one notebook.
- Playlist -> User. Each playlist belongs to one user.
- PlaybackQueue -> User. Each playback queue belongs to one user.
- AiConfig -> User. Each AI configuration belongs to one user.
- AiConversation -> User. Each AI conversation belongs to one user and stores a notebook UUID.

Many-to-Many:

- Playlist -> Notebooks. A playlist can contain many notebooks, and a notebook can appear in multiple playlists.
- PlaybackQueue -> Notebooks. A playback queue can contain many notebooks in a selected order.

Key Entities and Tables:

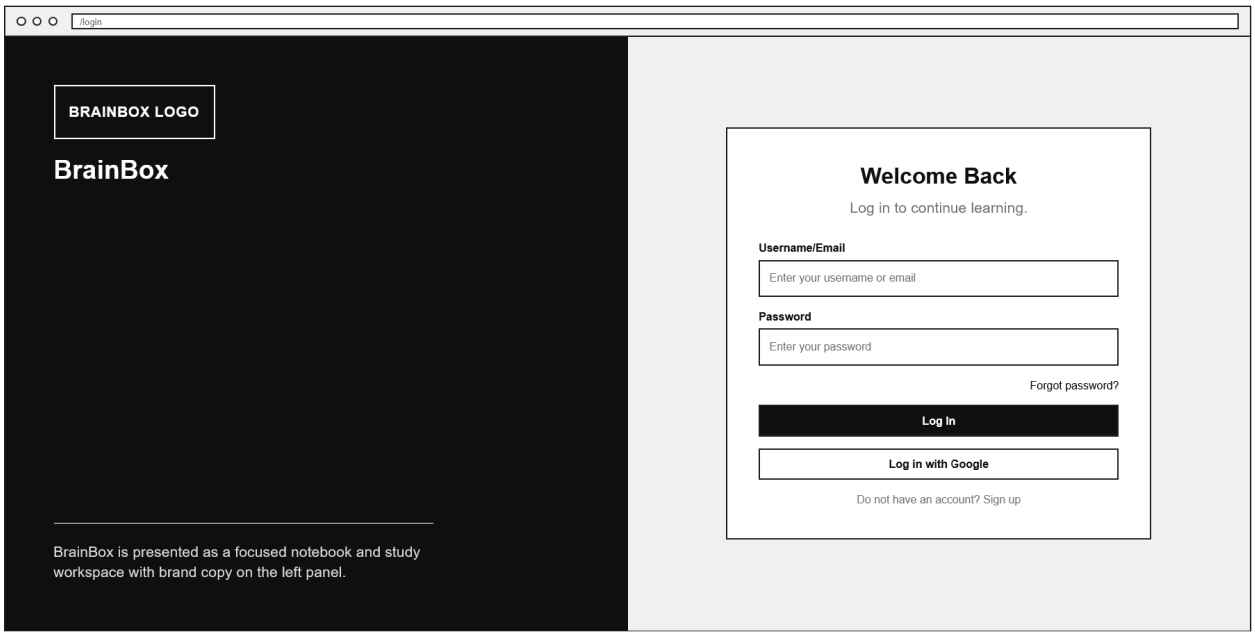
- `users` - Explicit table for user accounts, authentication fields, roles, verification state, provider identity, and selected AI config.
- `RefreshToken` / `Code` - Refresh token sessions and email verification/password reset codes.
- `Category` - User-owned notebook categories.
- `Notebook` - Notebook metadata, HTML content, owner, category, review time, and numeric version.
- `NotebookVersion` - Notebook content snapshots with timestamped version records.
- `notebook_mutation_record` - Explicit table for notebook mutation idempotency metadata.
- `Quiz`, `QuizQuestion`, `QuizAttempt` - Quiz metadata, ordered questions/options, and score attempts.

- `Flashcard`, `FlashcardCard`, `FlashcardAttempt` - Flashcard deck metadata, ordered cards, and mastery attempts.
- `Playlist` and `playlist_notebooks` - Ordered notebook playlists and join table positions.
- `PlaybackQueue` and `playback_queue_notebooks` - Current playback queue, selected playlist, and ordered notebook join data.
- `ai_config` - Explicit table for user-owned AI provider configuration.
- `ai_conversation` - Explicit table for AI conversation metadata and JSON/text messages by notebook UUID.

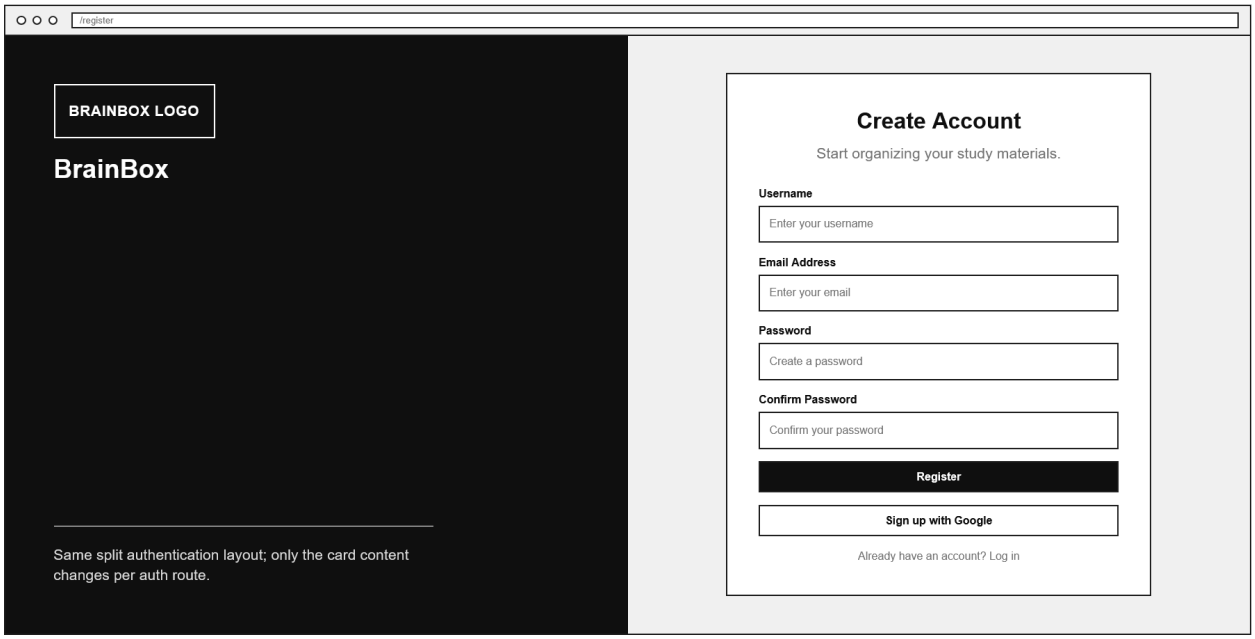
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

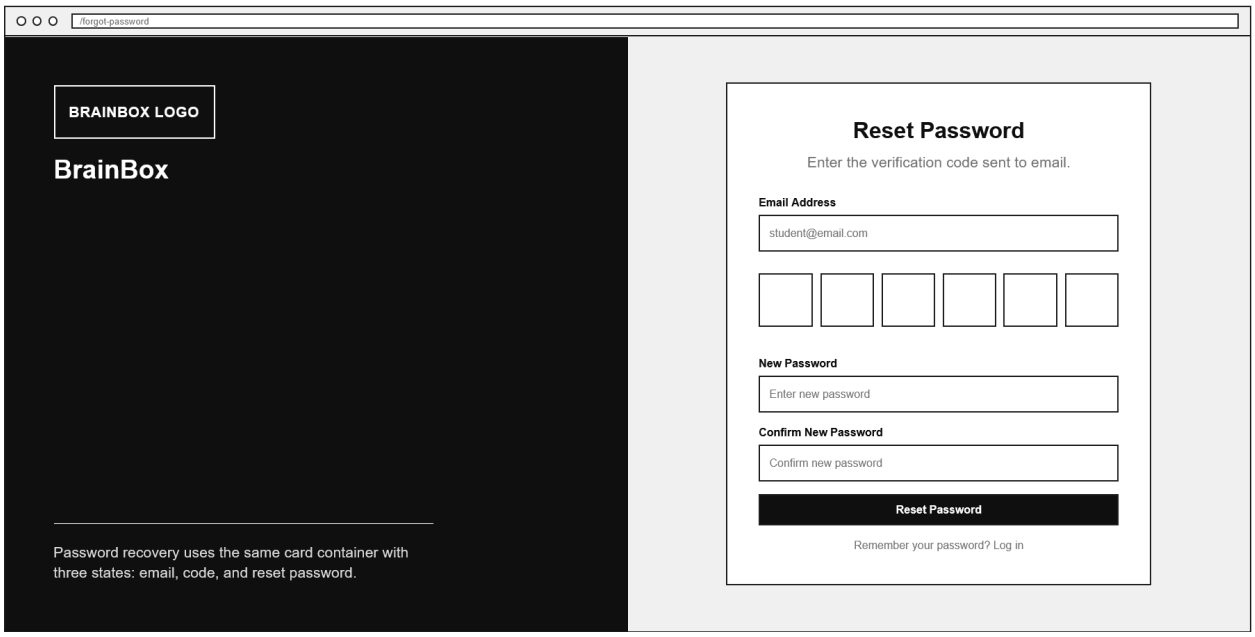
Login Screen



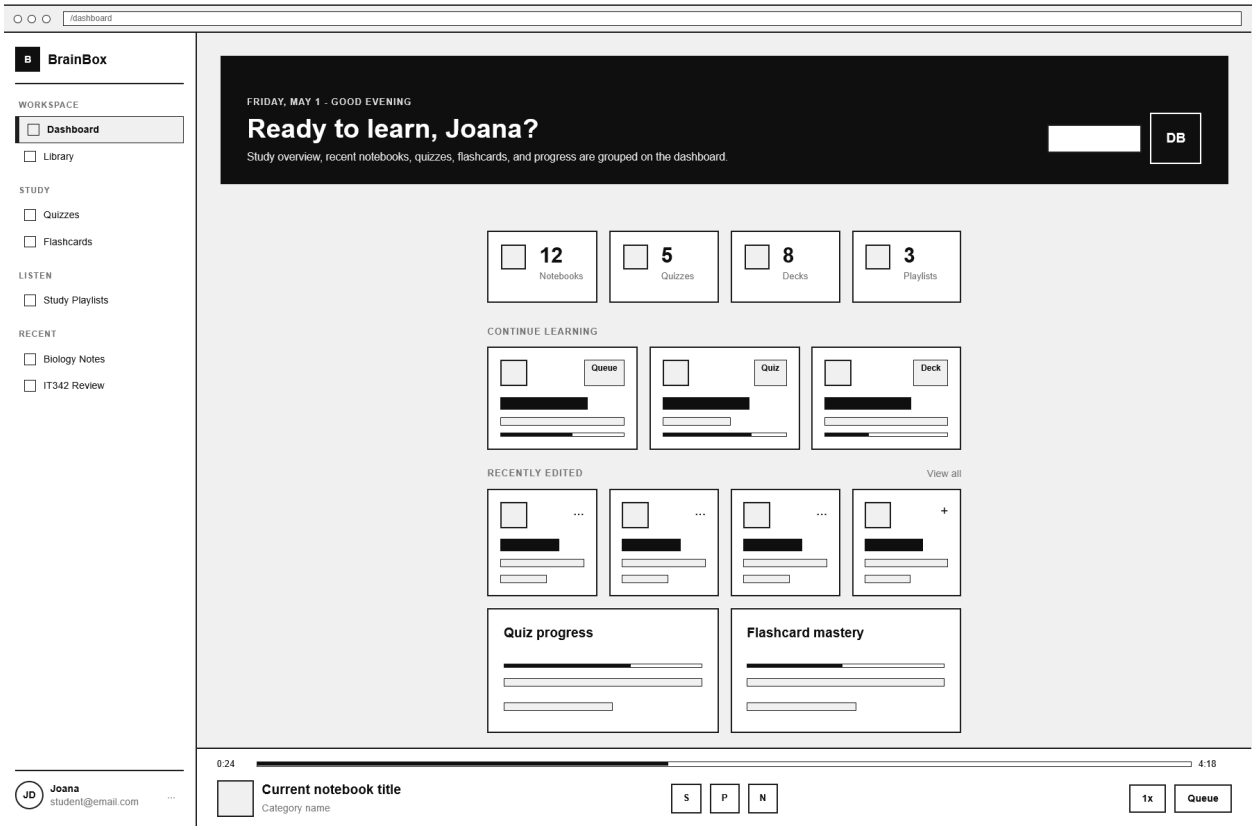
Register Screen



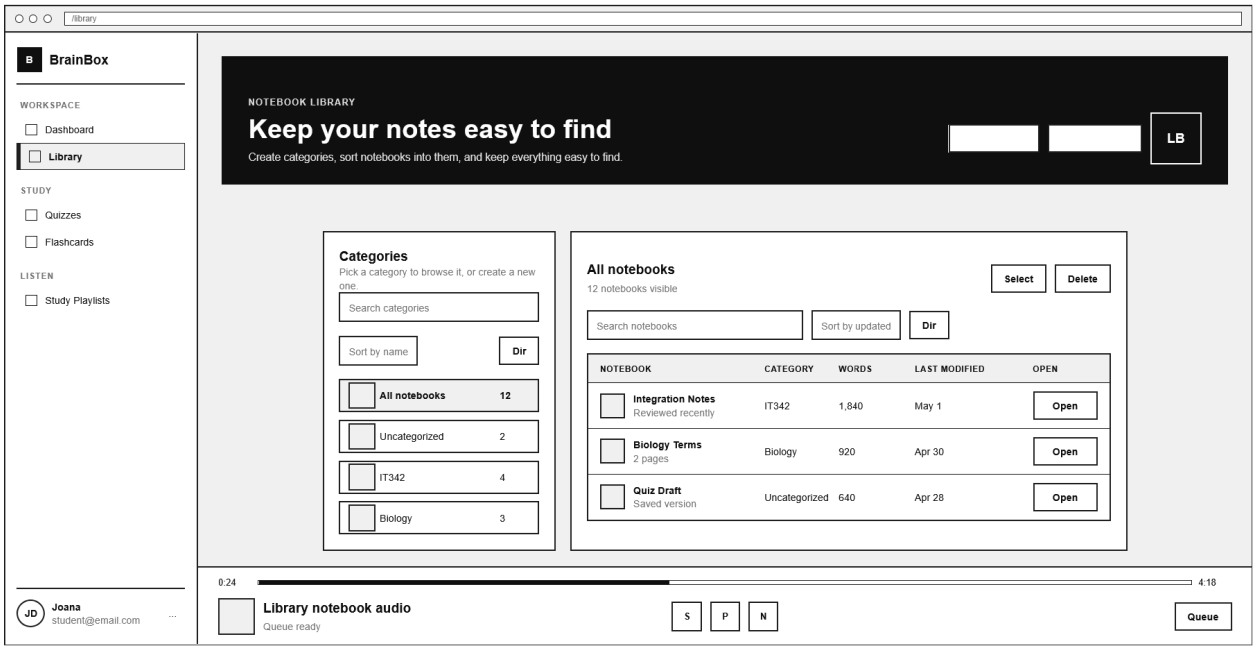
Forgot Password Flow



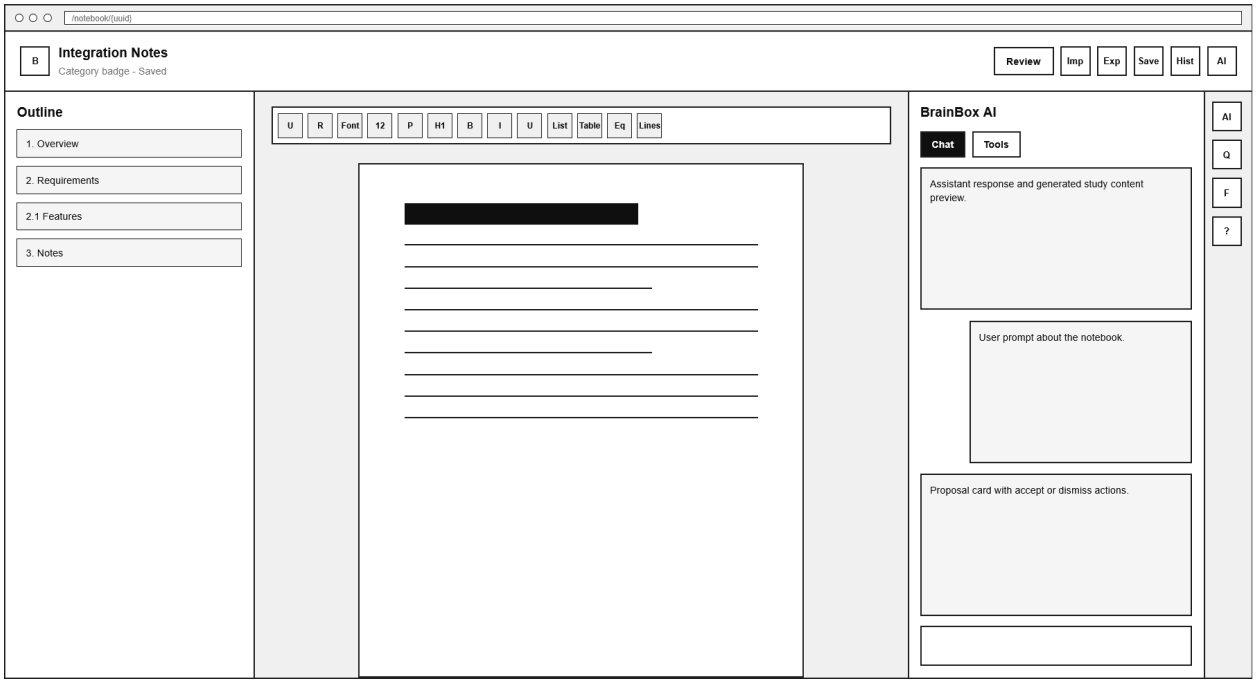
Dashboard



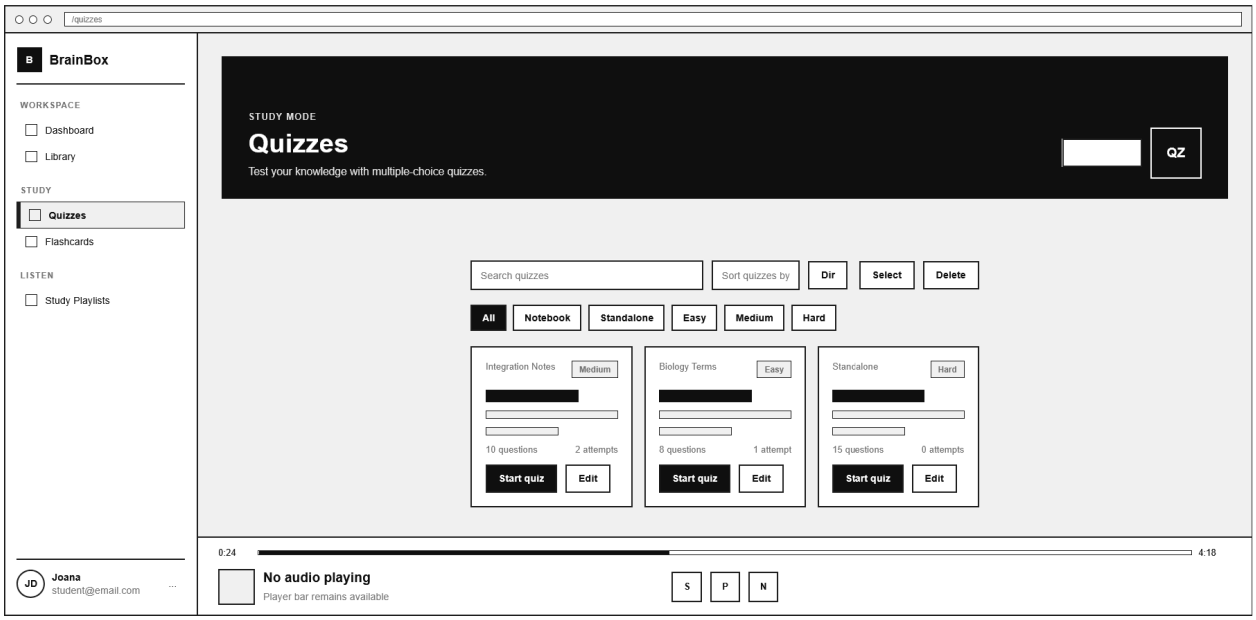
Library



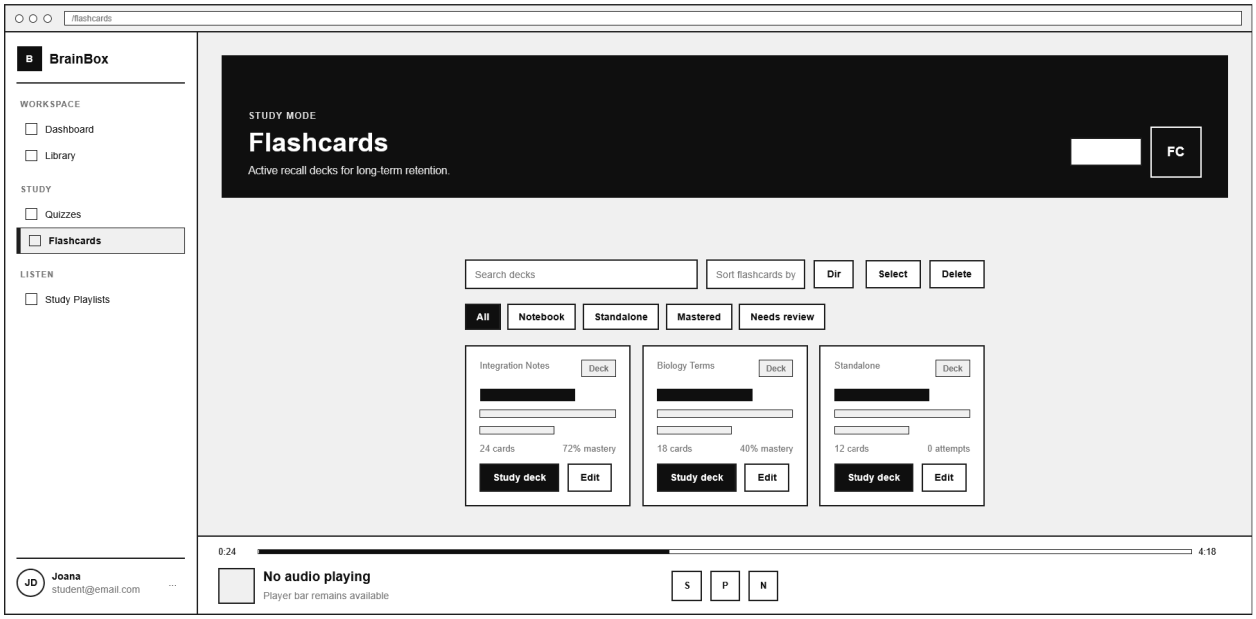
Notebook Editor with AI Sidebar



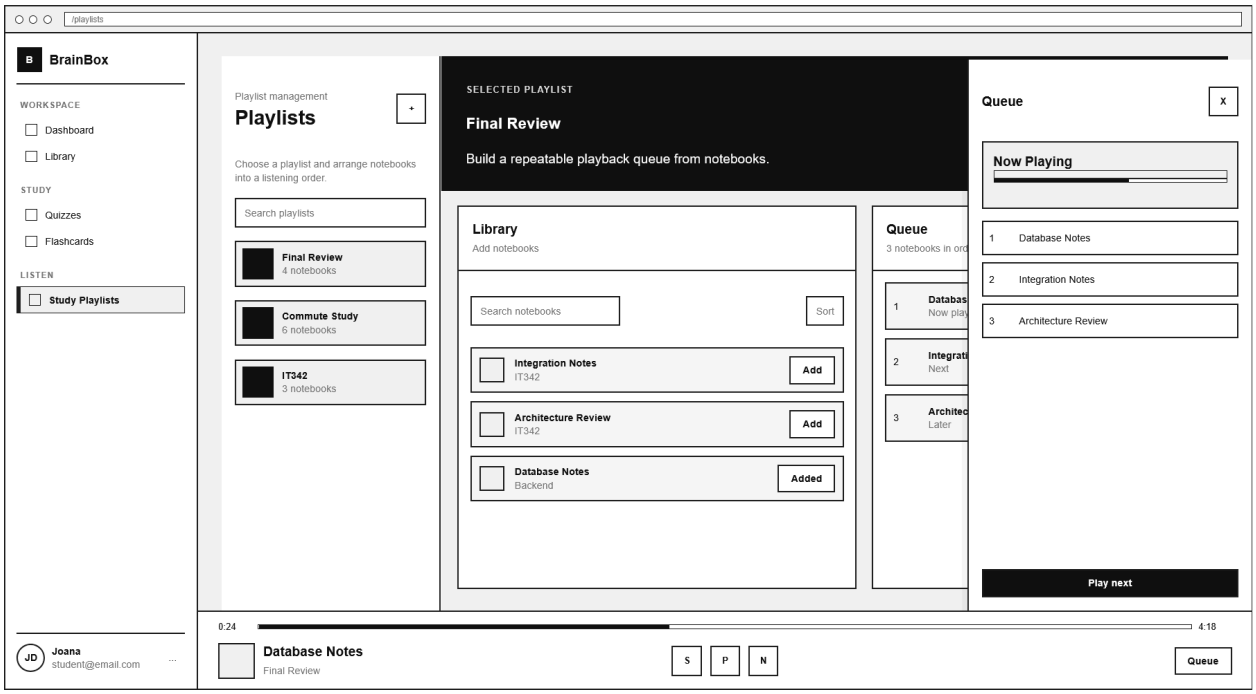
Quizzes



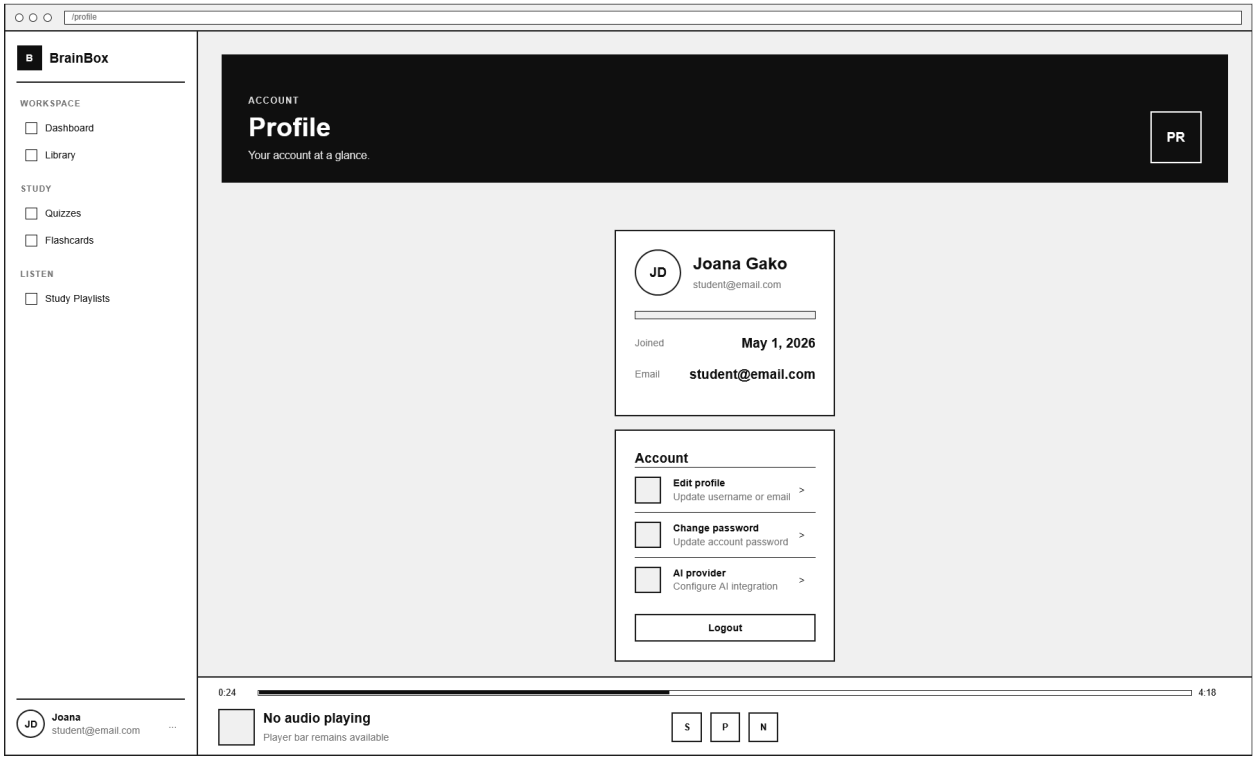
Flashcards



Playlists and Playback Queue

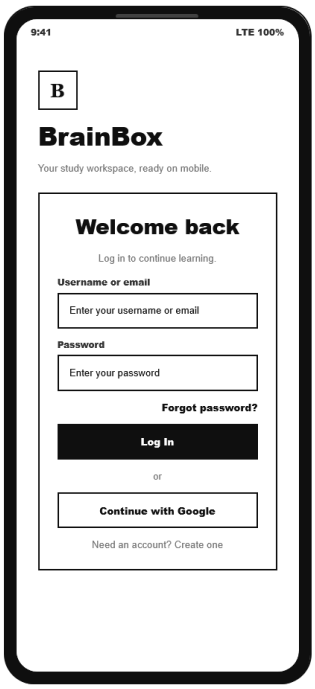


Profile

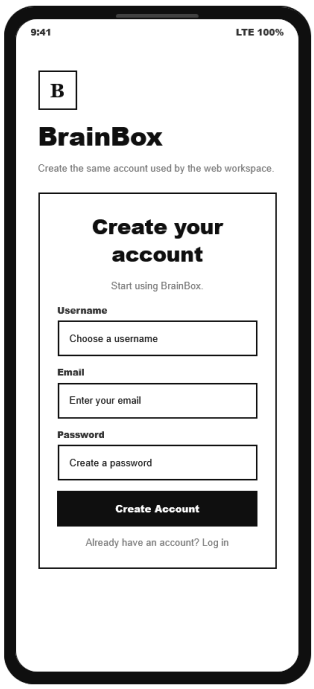


7.2 Mobile Application Wireframes

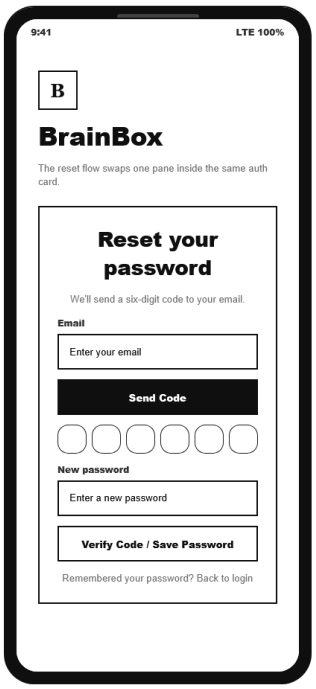
Mobile Login



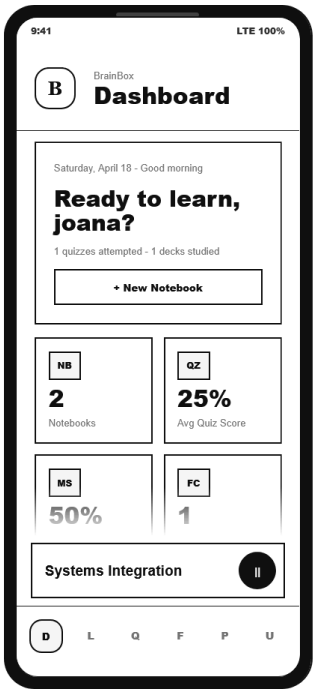
Mobile Register



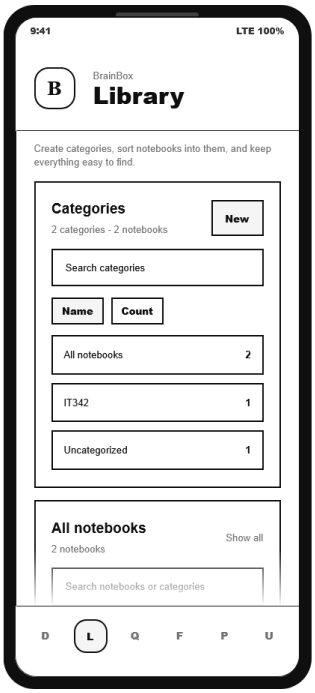
Password Reset Flow



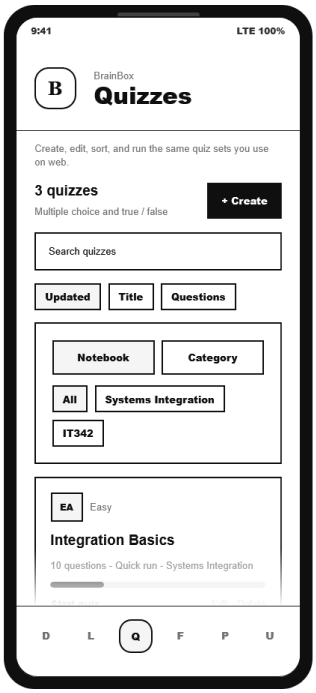
Dashboard Tab



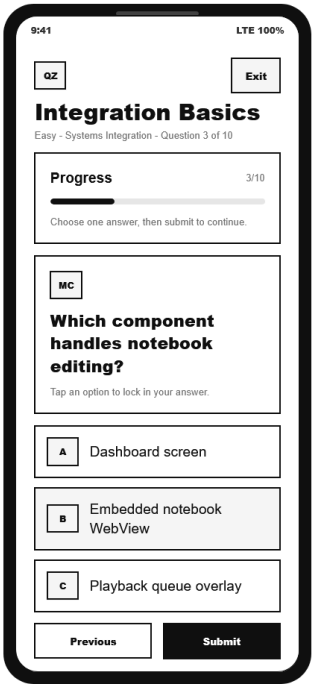
Library Tab



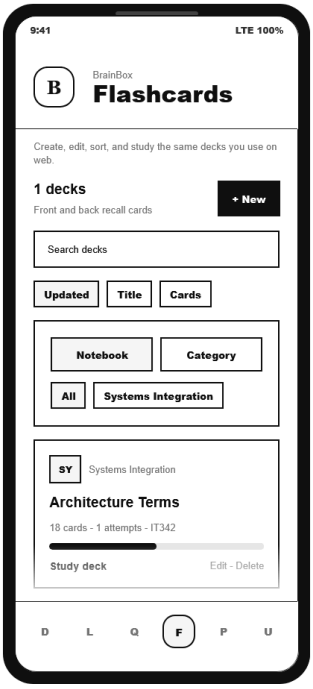
Quizzes Tab



Quiz Study Session

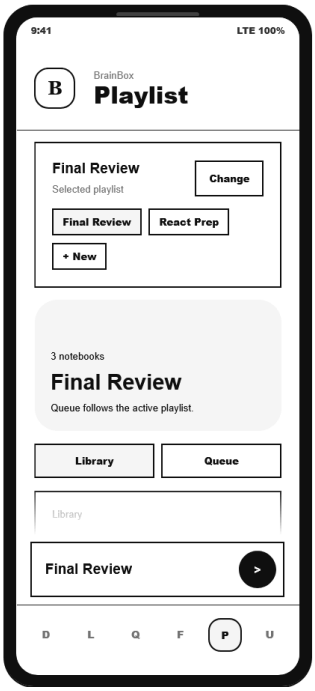


Flashcards Tab



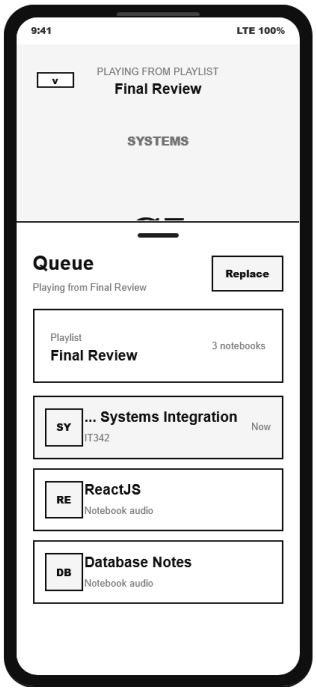
Flashcard Study Session

Playlist Tab

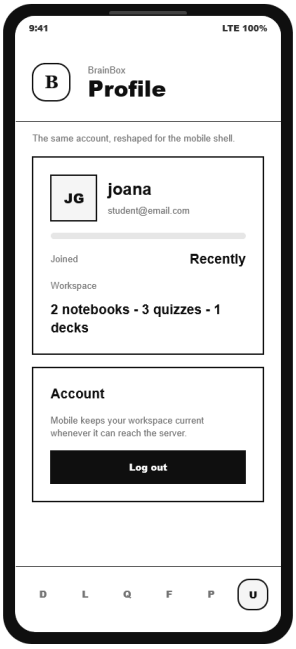


Notebook Editor WebView

Global Playbar and Queue



Profile Tab



8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation.

Day 3-4: Complete FRS/SRS and non-functional requirements.

Day 5-7: System architecture design.

Week 2: Detailed Design

Day 1-2: API specification.

Day 3-4: Database design.

Day 5-6: UI/UX wireframes.

Day 7: Implementation plan finalization.

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies.

Day 2: Database configuration and entities.

Day 3: JWT authentication implementation.

Day 4: User and profile endpoints.

Day 5: Notebook and category endpoints.

Week 4: Core Features

Day 1: Quiz endpoints.

Day 2: Flashcard endpoints.

Day 3: Notebook version history, restore, and conflict handling.

Day 4: Playlist and playback queue endpoints.

Day 5: AI configuration, conversation, validation, and API testing.

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React and Vite setup.

Day 2: Authentication pages.

Day 3: Dashboard and library pages.

Day 4: Notebook editor integration.

Day 5: API integration, editor version history, and state handling.

Week 6: Complete Web Features

Day 1: Quiz screens.

Day 2: Flashcard screens.

Day 3: Playlist and playback queue screens.

Day 4: Profile, AI settings, export, and review/version UI polish.

Day 5: Responsive design and web testing.

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android project setup and dependencies.

Day 2: Authentication screens.

Day 3: Home tabs and library screen.

Day 4: Notebook API integration and mobile library behavior.

Day 5: API service layer, token refresh, and playback service setup.

Week 8: Complete Mobile App

Day 1: Quiz study screen.

Day 2: Flashcard study screen.

Day 3: Playlist and playback controls.

Day 4: Mobile polish and integration testing.

Day 5: Emulator/device testing and APK generation.

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across backend, web, and mobile.

Day 2: Bug fixes and optimization.

Day 3: Security review.

Day 4: Performance and compatibility testing.

Day 5: Documentation updates.

Week 10: Deployment

Day 1: Backend deployment.

Day 2: Web application deployment.

Day 3: Mobile APK distribution.

Day 4: Final system testing.

Day 5: Project submission.

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system.
2. Notebook, category, version history, and restore management.
3. Quiz and flashcard study workflows.
4. Playlist and playback queue workflows.
5. Android playback service integration.
6. Cross-platform testing.

Risk Mitigation:

- Start with simplest working version of each feature
- Test AI integration points early to handle latency
- Maintain robust error handling for external LLM calls
- Focus on core notebook/review flow before secondary features